→ Previous videos:
* Basic interface
* How to write terms / thms
* Proof techs → FW, BW, subst., induction on number
* Lists → recursions, proving using structural induction on lists

→ This video :: Automation of proofs in Isabelle (Basic)

* Simplification
: Applies substitutions to a given goal
• Hope: It finishes the goal, or makes it easier for you
• E.g. length (xs @ ys) = length xs + length ys
x > y ⇒ (x^c - y) + c = (x + c) - y
• It applies the subst's from left to right
• As many times as possible
— until no simplification rule can be subst'd in the goal
• Does not always terminate
E.g. f x = g x, g x = f x, Loops forever
• It has by default a very large library of simp-rules
• @ length, etc
• Any recursive function F, f_simps are added automatically
• You can manually extend its simp-rules / remove things
using apply (simp add: rule_1 del: rule_2),
• Note = If you want to remove all facts from simpset, except for a specific fact f_1
apply (simp only: f_1)

→ * auto:-
• ≈ simplification + classical reasoning
↙ repetitive subst. ↘ repetitive FW + BW reasoning
• Uses the same simpset as the simplifier
• Has its own bank of FW / BW reasoning lemmas
• Recall :: to extent the simpset for the proof method "simp" we used apply (simp add / del / only :: rule_1 rule_2 ...)
• We can use the same to change the simplification rules we used by auto.
• To add / remove a lemma to the classical reasoning
— apply (auto intro: fact_1) :— Adds fact_1 to the BW
— apply (auto dest: fact_1) —— add fact_1 to the FW
• Recall :: FW reasoning is preferable for manual reasoning
• For automation :: BW reasoning is preferable
— Because it can help better with debugging
→ you usually are more confident about the conclusion you want to derive
— Start from conclusion, & figure out the assumptions using BW reasoning
• Note :: auto applies to all open goals unlike simp, subst, rule ...

* Notes ::
• When you prove a lemma "lem1: "x ∈ y ⇒ y ∈ x"" and you want it to always be used for simplification
lem1 [simp]: "x ∈ y ⇒ y ∈ x"
• for FW/BW lem1 (dest / intro): ...
• To deal with different categories of goals, there are predefined sets of simps; E.g. algebra_simps, field_simps
• Naturals :: to translate between e.g. Suc (Suc n) & n + 2 we use eval_nat_numerals

* force, fastforce
• More powerful than auto
• Bad: - they cannot stop in the middle like auto or simp
→ Not good for debugging
- Unlike auto, they only apply to the first goal !!!
• Use simp / intro / dest ...